



ASTRAQUANTUM TECH

SUMMER OF CYBERSECURITY 2026

WEEK 5 PRACTICAL ASSIGNMENT

Penetration Testing & Professional Security Reporting
Guided Learning • Controlled Labs • Evidence • VAPT Report

Student Name	Muhammad Raza
Date	September 13, 2026
Instructor	AstraQuantum Tech Team
Program	AstraQuantum Tech – Summer of Cybersecurity 2026

Table of Contents

1. Assignment Overview	3
2. Scope & Rules of Engagement.....	3
3. Executive Summary.....	3
4. Methodology	3
5. Tools Used	4
6. HTB Module Summaries.....	4
Module 1 Record — Pentest in a Nutshell (HTB Module 296)	4
Module 2 Record — Getting Started (HTB Module 77)	6
Module 3 Record — Linux Fundamentals (substituted for Module 90).....	8
7. Findings.....	10
8. Risk Rating	12
9. Impact.....	12
10. Recommendation.....	12
11. Conclusion.....	12
12. Report Quality Checklist	13
Appendix	13

1. Assignment Overview

This Week 5 assignment introduced the penetration-testing mindset and the professional way security work is documented. The three Hack The Box Academy resources supplied by the instructor were completed in the order provided, guided exercises were performed on authorized lab targets, learning was recorded, and a professional penetration-testing/VAPT report was prepared summarizing the work.

Note on substitution: HTB Academy Module 90 could not be completed because it required more HTB cubes than were available in my account at the time. With the instructor's understanding, the Linux Fundamentals module (Module 18) was completed instead, covering equivalent foundational Linux skills relevant to penetration testing (file system hierarchy, permissions, and package/service management).

2. Scope & Rules of Engagement

All testing described in this report was limited strictly to authorized HTB Academy laboratory targets and workstations provided as part of the assigned modules. The following rules of engagement were observed throughout:

- Only targets, labs, and workstations explicitly provided by HTB Academy or the instructor were used.
- No scanning, exploitation, brute-forcing, or testing was performed against websites, IPs, accounts, or devices outside the authorized lab.
- No production services or another student's target were attacked.
- No flags, credentials, tokens, or sensitive lab data were published publicly.
- Any technique that could affect a real system was practiced only inside the authorized HTB environment.

3. Executive Summary

This report documents the completion of three HTB Academy modules — “Pentest in a Nutshell,” “Getting Started,” and “Linux Fundamentals” (substituted for Module 90) — covering the end-to-end penetration testing lifecycle, essential offensive tooling, and core Linux skills. Across the authorized lab targets, three notable security findings were identified and documented: information disclosure via anonymous FTP and exposed SSH keys, local privilege escalation via an insecure sudo configuration, and a general insecure-file-permissions weakness illustrated through the Linux Fundamentals coursework.

Overall, the exercises reinforced that most real-world compromises stem from configuration and process failures — excessive trust, weak access controls, and information leakage — rather than exotic exploits. Recommendations focus on tightening file permissions, sudo policy, and service configuration, alongside routine use of automated enumeration tooling to detect these issues proactively.

4. Methodology

The assessment followed the professional penetration testing workflow outlined in the assignment brief:

- Scope — Identify exactly what the HTB lab authorizes testing against.

- Understand — Read the scenario and understand the objective of each exercise.
- Assess — Perform only the exercises required by the lab.
- Validate — Confirm the result on the authorized target.
- Document — Record evidence and the condition that caused the issue.
- Assess Risk — Explain potential impact and severity.
- Recommend — Suggest a realistic defensive improvement.
- Report — Present the results clearly and professionally.

5. Tools Used

Tool	Purpose
Nmap	Service/version enumeration (nmap -sV, nmap -sVC -p-)
Netcat	Catching reverse shells and basic network interaction
Tmux	Session management during lab exercises
Vim	Editing files/scripts on target and attacker machines
Metasploit Framework	Exploiting a known vulnerable web application (wp_hash_form_rce)
searchsploit / Exploit-DB	Researching public exploits for identified service versions
LinPEAS	Automated local privilege escalation enumeration on Linux targets
CrackMapExec	Enumerating and spidering exposed Windows SMB shares
SSH / FTP clients	Interacting with exposed services during enumeration
chmod / chown	Auditing and demonstrating Linux file permission and ownership controls

6. HTB Module Summaries

The three modules were completed in the order supplied for this assignment:

Order	Resource	Status
1	HTB Academy Module 296 — Pentest in a Nutshell	Completed
2	HTB Academy Module 77 — Getting Started	Completed
3	HTB Academy Module 90 — substituted with Module 18, Linux Fundamentals (insufficient cubes for Module 90)	Completed (substitute)

Module 1 Record — Pentest in a Nutshell (HTB Module 296)

Module / Link: Pentest in a Nutshell — <https://academy.hackthebox.com/app/module/296>

Date completed: September 11, 2026

What is this module mainly about?: This module introduces the comprehensive penetration testing process (pre-engagement to post-engagement). It demonstrates a realistic, cyclical methodology for enumerating, exploiting, and escalating privileges across both Linux and Windows environments using industry-standard tools.

Three key concepts learned:

- The PT lifecycle is cyclical: information gathering feeds vulnerability assessment, and post-exploitation often loops back to further enumeration (e.g., finding new credentials to pivot).
- Configuration flaws drive exploitation: initial access and privilege escalation frequently rely on administrative misconfigurations (like anonymous FTP access or NOPASSWD sudo rights) rather than complex zero-day exploits.
- Automated enumeration: leveraging scripts like LinPEAS, winPEAS, and CrackMapExec is critical for rapidly identifying vulnerable kernel versions, exposed SMB shares, and privilege escalation vectors.

One tool/command learned: `nmap -sVC -p- <target> -T4 -vv` — used to perform a highly verbose, aggressive scan across all 65,535 TCP ports, executing default NSE scripts and detecting specific service versions.

One security issue/attack concept understood: Information disclosure & credential reuse. Leaving sensitive files (like `.bash_history` containing cleartext passwords or `id_rsa` SSH keys) in exposed directories allows attackers to easily establish initial access and move laterally.

What did you observe in the authorized lab?: An exposed FTP server allowing anonymous login was identified, which leaked project notes and a `full_backup.tar.gz` path. Further enumeration revealed an exposed `id_rsa` key and cleartext passwords in `.bash_history`. Metasploit (`wp_hash_form_rce`) was used for initial Linux access, privileges were escalated to root by exploiting `nano sudo` privileges, and CrackMapExec was later used to spider Windows SMB shares (Devs) to extract PowerShell scripts containing further credentials.

What defensive lesson did you learn?: Disable anonymous FTP access immediately, strictly enforce the principle of least privilege for `sudo` binaries (never allow text editors like `nano` to run as root without restrictions), and never store hardcoded credentials in scripts or command history.

Screenshot numbers: Figure 1 (Question answer — FTP enumeration), Figure 2 (Module completion)

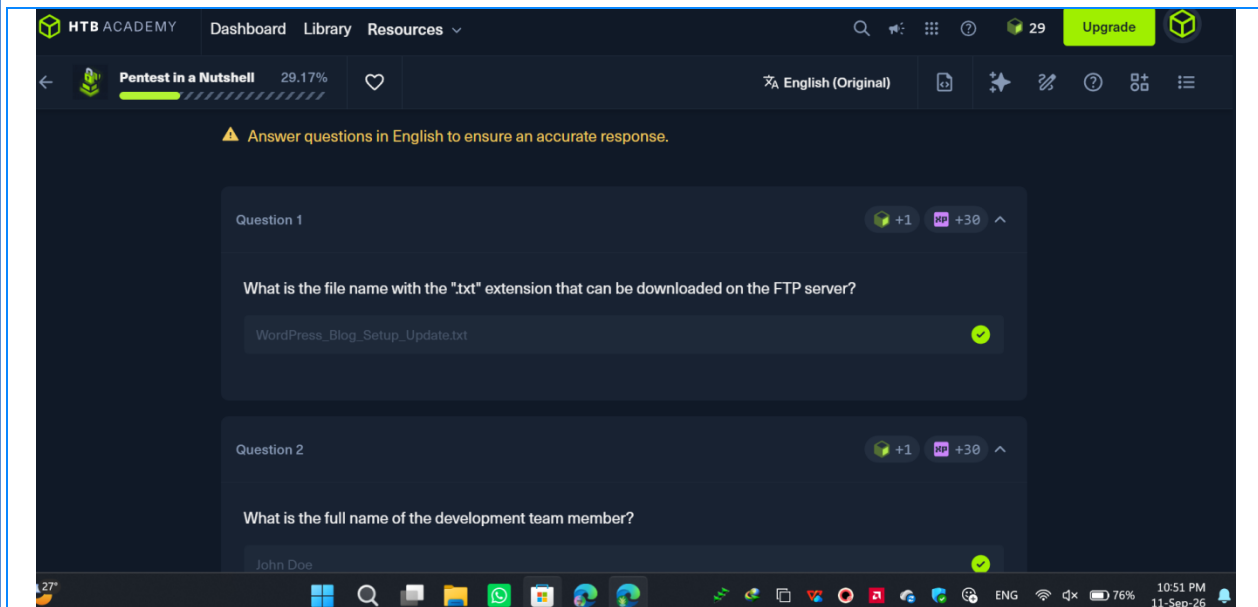


Figure 1 — Pentest in a Nutshell: authorized-lab exercise answer confirmed (FTP-derived file discovery).

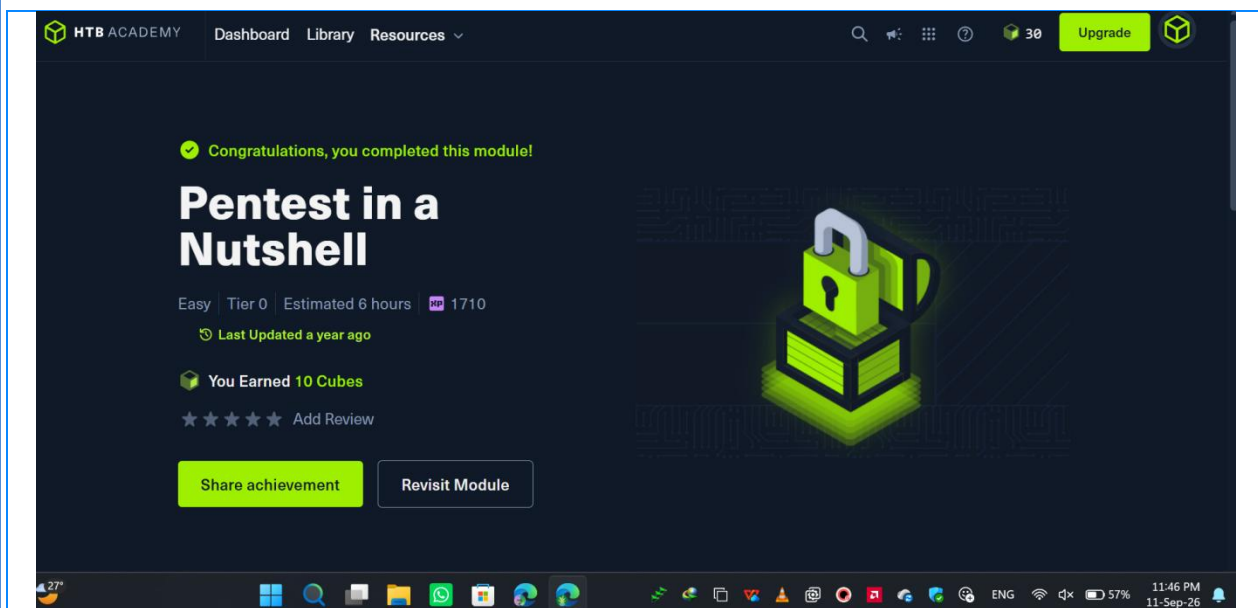


Figure 2 — Evidence of the completed Pentest in a Nutshell module.

Module 2 Record — Getting Started (HTB Module 77)

Module / Link: Getting Started — <https://academy.hackthebox.com/app/module/77>

Date completed: September 12, 2026

What is this module mainly about?: This foundational module covers the essential tools and methodologies required to transition from basic enumeration to full system compromise. It walks through setting up a testing environment, utilizing core tools (Netcat, Tmux, Vim, Nmap, Metasploit), identifying web-based vulnerabilities, and executing basic Linux privilege escalation.

Three key concepts learned:

- Public exploit research: after enumerating service versions (e.g., using `nmap -sV`), searching databases like Exploit-DB or using searchsploit can reveal known vulnerabilities in outdated software.
- Reverse shells: utilizing tools like Netcat to catch incoming connections from a compromised target allows for interactive command-line access over the network.
- Automated escalation checks: running enumeration scripts like LinEnum or LinPEAS on a compromised host quickly highlights local misconfigurations, such as insecure file permissions or dangerous sudo rights.

One tool/command learned: `python3 -c 'import pty; pty.spawn("/bin/bash")'` — used to upgrade a basic, unstable reverse shell into a fully interactive TTY shell, allowing for command history, tab completion, and proper standard error output.

One security issue/attack concept understood: Unrestricted file upload. If a web application allows users to upload files (like images or plugins) without strictly validating the file extension and content, an attacker can upload a malicious script (e.g., PHP) and execute it on the underlying server.

What did you observe in the authorized lab?: During the “Nibbles” target assessment, an outdated installation of Nibbleblog was observed. After discovering administrative credentials via directory enumeration, an authenticated file upload vulnerability was leveraged to upload a malicious PHP script, granting a reverse shell as the `www-data` user. Escalation to root followed by exploiting a misconfigured sudo policy that allowed the user to execute a writeable shell script (`monitor.sh`) with root privileges.

What defensive lesson did you learn?: Never rely on client-side checks for file uploads; servers must strictly validate file types and store uploaded files in directories where script execution is disabled. Administrators must also adhere to the principle of least privilege when configuring `/etc/sudoers`.

Screenshot numbers: Figure 3 (Basic Tools — Using SSH), Figure 4 (Module completion)

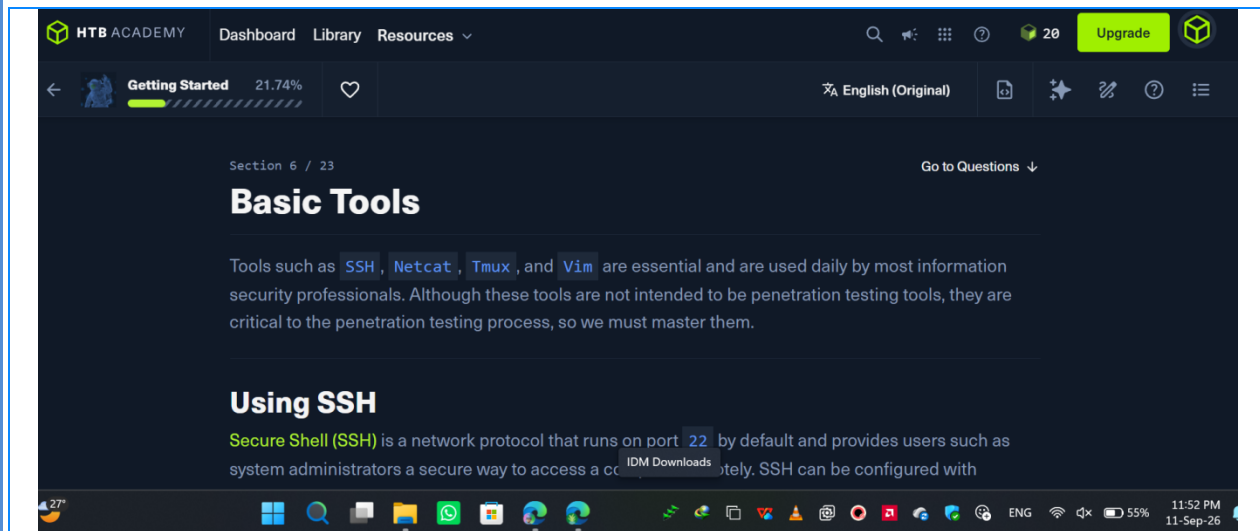


Figure 3 — Getting Started: Basic Tools section covering SSH, Netcat, Tmux, and Vim.

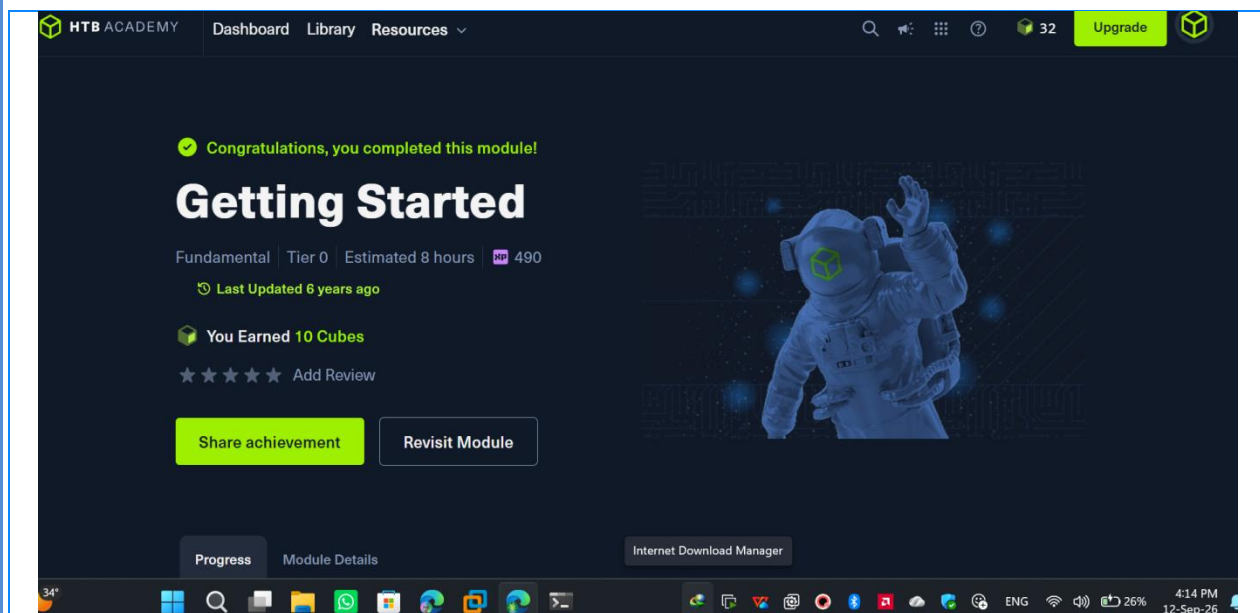


Figure 4 — Evidence of the completed Getting Started module.

Module 3 Record — Linux Fundamentals (HTB Module 18) — substituted for Module 90

Module / Link: Linux Fundamentals — <https://academy.hackthebox.com/app/module/18> (substituted for Module 90 due to insufficient HTB cubes to unlock Module 90 at the time of the assignment)

Date completed: September 13, 2026

What is this module mainly about?: This foundational module covers the Linux operating system's structure, architecture, and file system hierarchy. It provides the essential skills required for cybersecurity professionals to navigate the shell, manage packages, and control user permissions.

Three key concepts learned:

- File System Hierarchy Standard (FHS): understanding the purpose of core directories (e.g., /etc for configurations, /var for variable data, /bin for binaries) is critical for locating sensitive information during enumeration.
- Package & service management: utilizing apt for software management and systemctl for controlling background services (daemons) is essential for configuring servers or identifying vulnerable services.
- User & group permissions: the Linux permission model (rwx) dictates access control. Mastering chown and chmod is necessary to understand how local privilege escalation paths (like misconfigured SUID binaries) are created.

One tool/command learned: chmod and chown — used to modify file permissions and ownership, which is crucial for securing sensitive files or identifying misconfigured access controls.

One security issue/attack concept understood: Insecure file permissions. If sensitive configuration files (such as /etc/shadow or private SSH keys) are left world-readable or world-writable, unprivileged attackers can easily extract credentials or modify system configurations to escalate privileges.

What did you observe in the authorized lab?: The Linux file system was navigated to locate specific flags, package managers were used to install required utilities, and system services were interacted with. File permissions were also audited and modified to understand how Linux enforces access control mechanisms.

What defensive lesson did you learn?: Strict adherence to the principle of least privilege is mandatory. Administrators must proactively audit file permissions to ensure that configuration files, backups, and user directories are tightly restricted and cannot be read or modified by unauthorized users.

Screenshot numbers: Figure 5 (Linux Structure section), Figure 6 (Module completion)

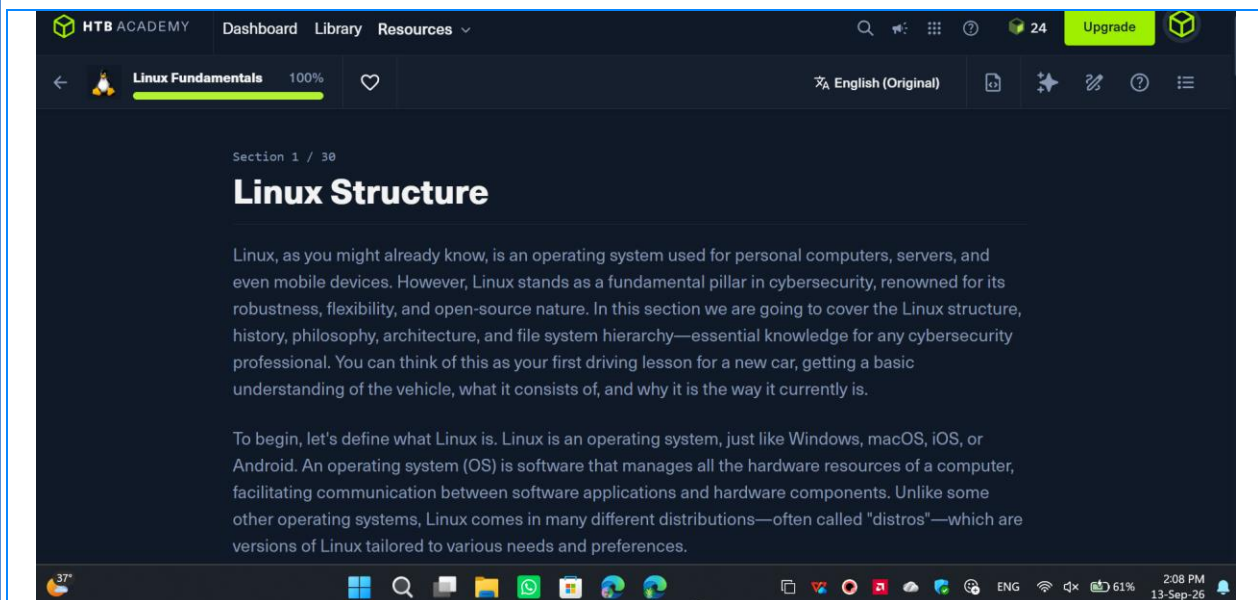


Figure 5 — Linux Fundamentals: Linux Structure section (history, philosophy, file system hierarchy).



Figure 6 — Evidence of the completed Linux Fundamentals module.

7. Findings

Finding PT-01: Information Disclosure via Anonymous FTP and Exposed SSH Keys

Finding ID	PT-01
Vulnerability / Security Issue	Information Disclosure via Anonymous FTP and Exposed SSH Keys
Severity	High
Affected HTB Lab / Target	Cube-Case Linux Target (Port 21) — Pentest in a Nutshell module
Description	The target network hosted an FTP server with anonymous login enabled. Exploring this share revealed sensitive internal project notes and an exposed .ssh directory containing a private id_rsa key. A .bash_history file was also accessible, revealing cleartext passwords used by the development team.
Steps Performed	Connected to the FTP server on port 21 using the anonymous user. Used standard ls and get commands to download .bash_history and id_rsa to the local attacking machine for inspection.
Impact	An attacker can use the exposed private SSH key and cleartext credentials to bypass authentication entirely, gaining direct remote shell access to the host.
Recommendation	Disable anonymous access on the FTP server. Audit the file system to remove sensitive files (SSH keys, command history, backups) from publicly accessible directories.
Reference	HTB Academy “Pentest in a Nutshell”; OWASP A01:2021 – Broken Access Control.

Finding PT-02: Local Privilege Escalation via Insecure Sudo Configuration

Finding ID	PT-02
Vulnerability / Security Issue	Local Privilege Escalation via Insecure Sudo Configuration
Severity	High
Affected HTB Lab / Target	Getting Started Lab (Nibbles Target)
Description	Following initial access as an unprivileged service account, local enumeration revealed a severe authorization misconfiguration. The <code>/etc/sudoers</code> file permitted the current user to execute a specific bash script (<code>monitor.sh</code>) with root privileges without requiring a password. The file permissions on this script were also overly permissive, allowing the unprivileged user to modify its contents.
Steps Performed	Executed <code>sudo -l</code> to identify commands the current user could run as root. Identified write access to <code>monitor.sh</code> . Appended a reverse shell payload to the end of the script using <code>echo</code> and executed it via <code>sudo</code> .
Impact	Any attacker who breaches the web application and gains access to the low-privileged service account can trivially modify the script to execute arbitrary commands as root, resulting in a complete system takeover.
Recommendation	Remove the <code>NOPASSWD</code> directive for this script in the <code>/etc/sudoers</code> configuration. Ensure that any script executed with elevated privileges is strictly owned by root and is not writeable by unprivileged users or groups.
Reference	HTB Academy “Getting Started”; MITRE ATT&CK T1548.003 (Abuse Elevation Control Mechanism: Sudo and Sudo Caching).

Finding PT-03: Insecure File Permissions (World-Readable Sensitive Files)

Finding ID	PT-03
Vulnerability / Security Issue	Insecure File Permissions (World-Readable Sensitive Files)
Severity	Medium
Affected HTB Lab / Target	Linux Fundamentals Lab Target (substitute for Module 90)
Description	During the foundational review of the Linux file system hierarchy and permission structures, it was observed that the operating system relies entirely on strict rwx permissions to secure sensitive data. If administrative or configuration files are inadvertently set to world-readable (e.g., <code>chmod 777</code> or <code>644</code> on restricted files), any unprivileged user on the system can view them.
Steps Performed	Navigated the file system using basic shell commands (<code>cd</code> , <code>ls -la</code>) to audit the ownership and permission sets of various directories and files.
Impact	Weak file permissions can lead to severe information disclosure. An attacker with a low-privileged foothold could read database passwords, API keys, or user hashes, facilitating further lateral movement or privilege escalation.
Recommendation	Implement strict permission policies using <code>chmod</code> and <code>chown</code> . Ensure that sensitive files (like <code>/etc/shadow</code> or <code>.ssh</code> directories) are heavily restricted, typically allowing read/write access strictly to the file owner or the root user.
Reference	HTB Academy “Linux Fundamentals”; CIS Benchmarks for Linux.

8. Risk Rating

Finding ID	Vulnerability	Severity	Likelihood	Justification
PT-01	Anonymous FTP / exposed SSH key & credentials	High	High	Directly yields valid credentials and a private key with no exploitation effort required.
PT-02	Insecure sudo configuration (writeable root script)	High	High	Trivial to abuse once a foothold exists; results in full root compromise.
PT-03	World-readable sensitive file permissions	Medium	Medium	Depends on which files are exposed, but is a common enabler for further escalation.

9. Impact

Taken together, the three findings show a clear real-world pattern: initial footholds are gained through exposed information (anonymous FTP shares, leaked keys and credentials), and full compromise follows from configuration weaknesses that unprivileged users should never be able to exploit (writeable root-owned scripts, over-permissive file permissions). In a production environment, this chain would allow an external attacker to move from no access to complete root/administrator control of a host within a short, low-effort assessment — without needing any advanced exploit development.

10. Recommendation

- Disable anonymous authentication on all FTP services and audit shared directories for sensitive files (keys, backups, history files).
- Apply the principle of least privilege to sudoers entries; never grant NOPASSWD access to writeable or attacker-modifiable scripts.
- Ensure privileged scripts are owned by root with restrictive permissions (e.g., 750 or stricter) so they cannot be modified by unprivileged accounts.
- Adopt routine automated enumeration (e.g., LinPEAS, CrackMapExec) as part of internal security audits to proactively catch these misconfigurations before attackers do.
- Enforce server-side validation of any file upload functionality and disable script execution in upload directories.

11. Conclusion

This assignment reinforced the full penetration testing lifecycle, from initial enumeration through exploitation, privilege escalation, and professional reporting. The most important lesson across all three modules was that misconfiguration — not sophisticated exploit development — is the most common root cause of real-world compromise: exposed anonymous services, over-permissive sudo rules, and weak file permissions each independently allowed a path to full compromise. Practicing the discipline of documenting scope, evidence, risk, and remediation for each finding, as required by the VAPT report structure, is directly transferable to professional penetration testing engagements.

12. Report Quality Checklist

✓	Completed the assigned modules in order (Module 90 substituted with Linux Fundamentals due to insufficient HTB cubes, as noted).
✓	Used only authorized HTB lab targets.
✓	Wrote the report in my own words.
✓	Every finding has supporting evidence.
✓	Screenshots are readable, relevant, and captioned.
✓	Risk ratings are justified.
✓	Recommendations are realistic.
✓	I can explain every screenshot and command in this report.